

1. Start here: choose the local setup

Insurance Policy Service | Step-by-step runbook | 26 September 2026

Follow pages 1-7 in order for the simplest persistent local setup. You need Node.js, the project and Postman. The local launcher starts MongoDB for you; an Atlas account is optional.

Step 1 - Check your tools

Open VS Code, then Terminal > New Terminal. Run each command below separately. Use Node.js 22 or 24 LTS for the versions covered by this project's CI. If node or npm is not found, install Node.js and reopen the terminal.

```
node --version
npm --version
git --version
```

Step 2 - Open the correct project folder

If you already have the repository, use VS Code > File > Open Folder and select its insurance-policy-service root. Open a terminal there. You should see package.json, src, postman and README.md. Otherwise, clone the repository using the commands below.

On another computer, open a terminal in your preferred parent folder, then run:

```
git clone https://github.com/Ankitkat30/insurance-policy-service.git
cd insurance-policy-service
```

Step 3 - Install the dependencies

```
npm ci
```

Wait until installation completes. This installs the exact dependency versions in package-lock.json, including the helper used to start local MongoDB. Internet access is required for installation and the initial MongoDB binary download. Do not use --omit=dev for npm run local.

All remaining terminal commands in this guide must run from the project root: the folder containing package.json. Commands shown here use macOS/Linux terminal syntax; PowerShell users can use Copy-Item instead of cp.

2. Create and understand the .env file

Step 4 - Create .env if it does not already exist

For npm run local, .env is optional: safe local defaults are built into the launcher. This guide creates it so you can see and control every setting. If .env already exists, edit it instead of overwriting your configuration.

```
cp .env.example .env
```

PowerShell equivalent: Copy-Item .env.example .env. Open .env in VS Code and use these values for the local walkthrough:

```
HOST=127.0.0.1
PORT=3000
MONGODB_URI=mongodb://127.0.0.1:27017/?replicaSet=rs0
DB_NAME=insurance_assessment
API_KEY=local-review-key-change-me
CPU_THRESHOLD=70
CPU_INTERVAL_MS=1000
SCHEDULER_INTERVAL_MS=1000
MAX_IMPORT_WORKERS=2
UPLOAD_DIR=./work/uploads
LOCAL_MONGO_PORT=27017
LOCAL_DATA_DIR=./work/mongodb
```

Step 5 - Save the file and understand the keys

HOST and PORT set the API address. DB_NAME chooses the database. LOCAL_MONGO_PORT and LOCAL_DATA_DIR choose the local MongoDB port and disk folder. UPLOAD_DIR holds temporary uploaded files.

CPU_THRESHOLD=70 triggers a server restart at the required CPU level. CPU_INTERVAL_MS=1000 samples every second. SCHEDULER_INTERVAL_MS=1000 checks for due messages about every second. MAX_IMPORT_WORKERS=2 limits simultaneous imports.

API_KEY protects /api requests. The demonstration value above matches the supplied Postman environment. It is not a MongoDB password or a key you obtain from a website. For your own deployment, choose a secret of at least 16 characters and update Postman to match.

MONGODB_URI is used by npm run dev/start. npm run local deliberately replaces it with the local database connection it provisions. Editing this URI alone does not make npm run local connect to Atlas. Save .env without adding it to Git; .gitignore already excludes it.

3. Start the server and view MongoDB

Step 6 - Start the persistent local application

```
npm run local
```

Keep this terminal open. On the first run, the launcher downloads a real MongoDB binary, starts a single-node replica set, then starts the supervised Node API. You do not need a separate mongod command.

Look for output similar to this. The data directory will include the full path on your computer:

```
MongoDB connection: mongodb://127.0.0.1:27017/  
?replicaSet=rs0&directConnection=true  
Database: insurance_assessment  
Persistent data directory: ../work/mongodb  
API ready: http://127.0.0.1:3000
```

The connection URI above is wrapped for display. Use the complete one-line URI below when connecting.

Step 7 - Verify the API is ready

Open <http://127.0.0.1:3000/health> in your browser, or use a second terminal:

```
curl http://127.0.0.1:3000/health
```

Expect HTTP 200 and JSON containing status: "ok", cpuPercent and uptimeSeconds. This route does not require an API key. The root URL / is not a homepage and may return 404.

Step 8 - Connect MongoDB Compass

If you want a visual database browser, open MongoDB Compass, choose a new connection, paste this URI, and connect:

```
mongodb://127.0.0.1:27017/?replicaSet=rs0&directConnection=true
```

Refresh and open insurance_assessment. After the import on page 4, inspect agents, users, accounts, lobs, carriers and policies. scheduledJobs and messages support scheduled delivery. On a fresh setup these collections may initially be empty.

Data is physically stored in work/mongodb inside this project. These are binary WiredTiger files; read documents through Compass or the API. Stopping the launcher preserves data. Local records do not appear automatically in MongoDB Atlas. Do not delete work/mongodb if you want to retain them.

4. Configure Postman and import a file

Step 9 - Import the Postman files

Open Postman and choose Import. Import both files from the project's postman folder:

```
postman/insurance-policy-service.postman_collection.json
postman/local.postman_environment.json
```

Select Insurance Local in the environment selector. Check baseUrl = http://127.0.0.1:3000, apiKey = local-review-key-change-me, and username = Alex Example. If you changed API_KEY in .env, put the same value in Postman's apiKey variable.

The collection supplies the x-api-key header. Postman web needs the separate Postman Desktop Agent running and connected to reach localhost. The Postman desktop application can send directly. Keep npm run local running too.

Step 10 - Run Health

Open GET Health and click Send. Expect HTTP 200 and status "ok". If it fails, fix connectivity before continuing.

Step 11 - Attach and send the CSV

Open POST Import CSV or XLSX. Choose Body > form-data. The checked row must have key file and type File. Click Select files and choose the actual examples/sample.csv file. Do not select its folder. Do not add extra text fields.

Do not manually set Content-Type: Postman adds multipart/form-data with the required boundary. Set the request timeout to at least 150000 ms if needed. Click Send and wait for the worker to finish.

```
{
  "processed": 3,
  "succeeded": 3,
  "rejected": 0,
  "errors": []
}
```

Expect HTTP 200. Repeat the upload to check that the same policies are updated instead of duplicated. For the assessment data, select your original local CSV instead; that file has 1,198 rows and is deliberately not committed to Git.

If you see EISDIR, Postman could not read the selected path as a file: reselect the actual CSV. If an import is interrupted by the required CPU restart, wait for /health to recover and re-upload; already committed rows can be safely repeated.

5. Search, aggregate and schedule messages

Step 12 - Search policies

Run GET Search policies by name or email. With username = Alex Example and the synthetic file imported, expect HTTP 200 and total: 2. Search is an exact case-insensitive first-name or email match. When using the assessment CSV, set username to a name or email that actually exists in it.

```
GET {{baseUrl}}/api/policies/search?username={{username}}
```

Step 13 - View each user's policy summary

Run GET Aggregate policies by user. Expect HTTP 200 with per-user policy counts, date ranges and category/carrier IDs. A fresh synthetic-only database has Alex with two policies and Taylor with one. Existing assessment data adds more users; totals need not stay at two.

```
GET {{baseUrl}}/api/policies/aggregate?page=1&limit=20
```

Step 14 - Schedule a message

Run POST Schedule message (one minute from now). Its pre-request script creates a future UTC date/time. The response script saves jobId. Expect HTTP 201 with status "pending". This is the simplest way to avoid accidentally submitting a time in the past.

For a manually composed request, use Body > raw > JSON and the fields below. Replace the example date/time with a future instant before sending. The collection script may overwrite day/time variables; use a separate manual request to control them yourself.

```
{
  "message": "Assessment reminder",
  "day": "2026-09-27",
  "time": "15:30:00",
  "timezone": "Asia/Kolkata"
}
```

Step 15 - Check delivery before and after the due time

Immediately run Get schedule status: it should be pending. After the due time, run it again: it should be delivered. Then run List delivered messages and Find delivery for scheduled job. The matching message should appear once.

day is YYYY-MM-DD, not a recurring weekday. time is 24-hour HH:mm or HH:mm:ss. UTC is used if timezone is omitted. A schedule is saved immediately in scheduledJobs; the actual message is inserted later in messages. Normal delivery has about a one-second polling delay plus workload/database latency.

6. Middleware: what you need to do

You do not run middleware separately, call a middleware URL, or install each middleware manually. npm ci installs its dependencies, and app.js/routes register the middleware when the server starts. Your role is to send requests in the required format.

Authentication - auth.middleware.js

This checks the x-api-key header on /api requests. Use the same key as API_KEY in .env. The supplied collection already sends {{apiKey}}. If you get 401 Invalid API key, fix the environment/header value, then Send again. /health intentionally needs no key.

```
Header name: x-api-key
Header value: local-review-key-change-me
```

With npm run local the launcher supplies the demo key if none is configured. With npm run dev/start authentication is optional unless API_KEY is set; this guide explicitly sets it to keep behavior clear.

Uploads and capacity - upload.middleware.js

This accepts one CSV/XLSX file under field name file, checks its extension, limits size to 10 MB, rejects unexpected text fields and checks available worker capacity. Use Body > form-data and type File. Do not send a local file path as raw JSON.

413 means the upload is too large. 400 can mean an unexpected field or unsupported file type. 503 at admission means import workers are busy; the response supplies Retry-After: 5. Wait and retry rather than starting many uploads.

Errors and missing routes - error.middleware.js

This turns failures into JSON responses. A 400 validation response identifies invalid input; read its details and correct the request. A 404 means the URL/method is wrong or the requested schedule does not exist. Unexpected internal failures are logged and return a generic error rather than a stack trace.

Built-in middleware in app.js

express.json reads JSON bodies with a 32 KB limit, so scheduling requests need Content-Type: application/json. Helmet sets security-related response headers automatically. A generated X-Request-Id response header helps match a failed request with a server log.

What next() means in the code

Middleware receives req, res and next. Calling next() means continue to the next handler. Sending a response such as res.status(401).json(...) ends that request. Failed authentication therefore prevents the controller and database query from running.

7. Stop, restart, test and troubleshoot

Step 16 - Stop and restart safely

In the terminal running `npm run local`, press `Ctrl+C` once and allow shutdown to finish. It stops the API and MongoDB but keeps `work/mongodb`. To resume later, open the same project and run `npm run local` again. You do not need `npm ci` on every start; use it after dependency changes or on a fresh checkout.

After editing `.env`, stop and restart the launcher so it reads the new values. Keep the same `LOCAL_MONGO_PORT` for an existing database directory: the stored replica-set configuration uses that address.

Step 17 - Run the automated checks

From the project root, use a second terminal or stop the application first. The tests create their own temporary databases and do not reset your local assessment database.

```
npm run format:check
npm run check
npm test
npm audit --omit=dev
```

Without `ASSESSMENT_SAMPLE` the private dataset test is skipped. To include it, replace the example path below with the full path to your original assessment CSV:

```
ASSESSMENT_SAMPLE='/absolute/path/to/assessment.csv' npm test
```

If something fails

`npm cannot find package.json`: your terminal is in the wrong folder. Open the `insurance-policy-service` root.
`node/npm not found`: install Node and reopen the terminal. `Dependency/binary download fails`: restore internet access and retry; do not remove database files.

`ECONNREFUSED` or `Could not send request`: confirm the API terminal is still running, then test `/health`. In Postman web, confirm Desktop Agent is connected. A Postman offline banner can also indicate its own workspace connection problem.

`Port 3000 or 27017 already in use`: do not start another copy. Use the already-running service or stop its terminal first. If you deliberately change `PORT`, also change Postman `baseUrl`. Do not run Docker MongoDB and `npm run local` together on 27017.

`Search returns no records`: import first, verify the chosen database, and use an exact name/email. `Schedule returns 400`: use a future date/time and valid timezone. `Message is not yet delivered`: check `scheduledAt` and the `scheduler/server` state. The required CPU restart can briefly interrupt service; wait for readiness and retry.

8. Optional alternatives and reviewer checklist

Alternative A - Docker Compose

If Docker Desktop is installed and running, stop `npm run local` first, then run from the project root:

```
docker compose up --build
```

This starts both MongoDB and the API. An `.env` file is not required with the supplied defaults; Compose uses `API_KEY` from `.env` if present. Its default key is the same demo key used in this guide. The current Compose configuration explicitly supplies its own MongoDB URI; other `.env` values do not automatically replace every Compose setting.

Stop foreground Compose with `Ctrl+C`. `docker compose down` removes its containers but preserves the named `mongo-data` volume. Do not add `-v` if you need the data: that removes the volume. Docker data and `work/mongodb` are separate stores; switching startup methods does not migrate records.

Alternative B - Your own MongoDB replica set or Atlas

Use this only when you already have a working database connection. Create/edit the ignored `.env` and set `MONGODB_URI` to that database's URI, `DB_NAME` to your chosen database and `API_KEY` to your own secret. For Atlas, your database user and network access must permit the connection. Keep the URI/password private.

```
npm ci
# Edit .env with your external database connection
npm run dev
```

`npm run dev` reads `.env` and starts only the supervised API. It does not start MongoDB for you. A standalone MongoDB server is insufficient: this project requires replica-set transactions. `LOCAL_DATA_DIR` and `LOCAL_MONGO_PORT` are only used by `npm run local`.

Choose the correct command

`npm run local` = persistent development MongoDB plus API; `.env` optional. `npm run dev` = API using `.env` and an existing database; `.env` required. `npm start` = API using process environment variables; it does not load `.env` automatically. `npm run demo` = temporary MongoDB plus API; data is removed on shutdown.

Final reviewer checklist

1. Clone/open the project and run `npm ci`. 2. Follow the local `.env` setup or use documented defaults. 3. Run `npm run local`. 4. Check `/health`. 5. Import the Postman collection/environment. 6. Upload examples/sample.csv. 7. Run search and aggregation. 8. Schedule a message and verify delivery. 9. Run checks. 10. Share the GitHub link without `.env`, database files or personal CSV records.

No manual middleware startup is needed. The local path does not require an Atlas account. Repository: <https://github.com/Ankitkat30/insurance-policy-service>